

Blocco 7 - Gentle Introduction: DDL, DML e transazioni

Creare oggetti, modificare dati e verificare prima del commit

Relational Databases & SQL

Materiale aggiuntivo

Perché questo materiale

Una introduzione progressiva alla sintassi e al metodo del blocco 7.

Problema di partenza

Situazione concreta

Finora abbiamo letto dati. Ora dobbiamo creare tabelle, inserire righe, aggiornare valori e annullare modifiche non corrette.

Ponte con i blocchi precedenti

Le query dei blocchi precedenti diventano strumenti di controllo prima e dopo una modifica.

Idea guida

Ogni modifica deve essere trattata come una procedura: seleziono, modifico, verifico, confermo o annullo.

- ▶ prima si scrive la domanda in linguaggio naturale;
- ▶ poi si decide il soggetto della query;
- ▶ poi si dichiara la granularità del risultato;
- ▶ solo alla fine si rifinisce la sintassi.

Sintassi essenziale

La sintassi è utile solo se rappresenta una decisione chiara sul significato delle righe.

Schema sintattico

```
BEGIN;  
  
SELECT ... FROM tabella WHERE condizione;  
  
UPDATE tabella  
SET colonna = nuovo_valore  
WHERE condizione  
RETURNING colonne;  
  
ROLLBACK;
```

Come leggere la sintassi

- ▶ `CREATE TABLE` definisce una struttura.
- ▶ `INSERT` aggiunge righe.
- ▶ `UPDATE` modifica righe esistenti.
- ▶ `DELETE` cancella righe.
- ▶ `BEGIN`, `COMMIT`, `ROLLBACK` delimitano una transazione.

Ciclo sicuro di una modifica

- ➊ Leggere le righe candidate con una `SELECT`.
- ➋ Controllare conteggio e valori.
- ➌ Aprire una transazione con `BEGIN`.
- ➍ Eseguire `INSERT`, `UPDATE` o `DELETE`.
- ➎ Usare `RETURNING` per vedere cosa è cambiato.
- ➏ Concludere con `ROLLBACK` in test o `COMMIT` se la modifica deve restare.

DDL: progettare prima di creare

- ▶ Il tipo della colonna limita i valori ammessi.
- ▶ PRIMARY KEY identifica una riga.
- ▶ REFERENCES mantiene il collegamento con una tabella padre.
- ▶ CHECK rende esplicite regole di dominio.
- ▶ DEFAULT assegna un valore quando l'utente non lo specifica.

DDL: esempio con vincoli

```
CREATE TABLE IF NOT EXISTS product_price_audit (  
  audit_id bigserial PRIMARY KEY,  
  product_id integer NOT NULL REFERENCES products(product_id),  
  old_price numeric(10, 2) NOT NULL CHECK (old_price >= 0),  
  new_price numeric(10, 2) NOT NULL CHECK (new_price >= 0),  
  reason text NOT NULL,  
  changed_at timestamp NOT NULL DEFAULT now()  
);
```

Dal linguaggio naturale a SQL

Una query è una frase controllabile: soggetto, condizioni, trasformazioni e risultato. Se una parte della frase resta ambigua, anche il codice diventa fragile.

- ▶ il soggetto decide da dove partire;
- ▶ le condizioni decidono cosa includere;
- ▶ le trasformazioni decidono cosa calcolare;
- ▶ l'ordinamento decide come leggere il risultato.

Esempi progressivi

Ogni esempio parte da una richiesta informale, passa per una specifica e arriva alla soluzione SQL.

Esempio 1: Creare una tabella di audit

Domanda

Creare una tabella di audit.

Controllo

Prima di leggere il codice, dichiarare popolazione, granularità e colonne finali attese.

Esempio 1: implementazione

```
CREATE TABLE IF NOT EXISTS product_price_audit (  
    audit_id bigserial PRIMARY KEY,  
    product_id integer NOT NULL REFERENCES products(product_id),  
    old_price numeric(10, 2) NOT NULL,  
    new_price numeric(10, 2) NOT NULL,  
    changed_at timestamp NOT NULL DEFAULT now()  
);
```

Esempio 2: Aggiornamento controllato con rollback

Domanda

Aggiornamento controllato con rollback.

Controllo

Prima di leggere il codice, dichiarare popolazione, granularità e colonne finali attese.

Esempio 2: implementazione

```
BEGIN;  
  
SELECT product_id, sku, unit_price  
FROM products  
WHERE category = 'accessories' AND active = true  
ORDER BY product_id;  
  
UPDATE products  
SET unit_price = round(unit_price * 1.05, 2)  
WHERE category = 'accessories' AND active = true  
RETURNING product_id, sku, unit_price;  
  
ROLLBACK;
```


Esempio 3: Inserimento controllato

Domanda

Inserimento controllato.

Controllo

Prima di leggere il codice, dichiarare popolazione, granularità e colonne finali attese.

Esempio 3: implementazione

```
BEGIN;  
  
INSERT INTO products (product_id, sku, product_name, category, unit_price, active)  
VALUES (1000, 'LAB-SVC', 'Laboratory Service', 'services', 250.00, true)  
RETURNING product_id, sku, product_name;  
  
ROLLBACK;
```

Esempio 4: Cancellazione controllata

Domanda

Cancellare una riga di test solo dopo averla identificata.

Controllo

Il filtro del DELETE deve essere prima provato come SELECT.

Esempio 4: implementazione

```
BEGIN;  
  
SELECT product_id, sku  
FROM products  
WHERE sku = 'LAB-SVC';  
  
DELETE FROM products  
WHERE sku = 'LAB-SVC'  
RETURNING product_id, sku, product_name;  
  
ROLLBACK;
```

Esempio 5: Da test a modifica definitiva

Domanda

Quando usare COMMIT?

- ▶ in laboratorio si usa quasi sempre ROLLBACK;
- ▶ in produzione si usa COMMIT solo dopo controlli espliciti;
- ▶ il commit non è una formula finale: è una decisione.

Esercizio guidato

Data una rappresentazione tabellare, costruire la query che risponde alla richiesta.

Rappresentazione tabellare

product_id	sku	category	unit_price
4	KEY-MECH	accessories	99.00
5	MOU-WL	accessories	39.00
7	BAG-15	accessories	79.00

Problema informale

Richiesta

Data la tabella prodotti, costruire una procedura SQL che aumenti del 5

Metodo

Evidenziare prima la granularità del risultato, poi scrivere la query.

- ▶ eseguire UPDATE o DELETE senza WHERE;
- ▶ non contare le righe candidate prima della modifica;
- ▶ fare test senza transazione;
- ▶ dimenticare vincoli di chiave e vincoli di dominio;
- ▶ non usare RETURNING quando serve vedere cosa è cambiato.

Checklist finale

- ❶ La domanda informale è stata tradotta in specifica?
- ❷ La granularità del risultato è dichiarata?
- ❸ I filtri e i collegamenti sono motivati?
- ❹ I nomi delle colonne finali sono leggibili?
- ❺ Esiste almeno una query di controllo?

- ▶ scrivere SQL significa rendere verificabile una domanda;
- ▶ ogni costrutto sintattico deve corrispondere a una scelta di modello;
- ▶ esempi piccoli e controlli intermedi evitano errori difficili da vedere.